World Cup Analytics Capstone

Step 0 — Setup Guide: creating your cloud database before touching any code

---

Read this whole guide before opening any script. Every later file (`ingest_raw.py`, the SQL scripts, the Power BI report) assumes the steps here are already done. Skipping ahead just means coming back to this guide anyway, once something fails to connect.

What you'll have by the end of this guide:

- A free Supabase account and organization
- A cloud Postgres database (your own project, empty and ready)
- A project folder on your computer, with a `.env` file holding your database credentials

---

Project Overview — read before you begin

The data sources

Two real, public datasets — not scraped. **jfjelstul/worldcup** (GitHub) covers every men's and women's World Cup, 1930–2022, across 22 files. **mominullptr's FIFA-World-Cup-2026-Dataset** covers 2026 only, across 10 files, but far more richly (expected goals, market value, ELO ratings). A third, tiny source is added by hand: 6 verified 2026 award winners, since neither dataset covers awards for 2026.

The plan: ELT, not ETL

Three tools, three separate jobs. **Python** downloads both sources and lands them untouched. **SQL**, running inside the cloud database, does all the cleaning, filtering, and modeling. **Power BI** connects live to the modeled tables and builds the report. Landing raw data first, before cleaning it, means every transformation decision stays visible and re-runnable in SQL, instead of happening invisibly during import.

Known incompleteness (accepted on purpose)

No football dataset is complete, and this project doesn't hide that. **Fouls** exist only for 2026 — older tournaments never tracked them. **Cards** exist only from 1970 onward, because the card system itself wasn't introduced before then. **Awards** vary by year, since FIFA introduced the Golden Ball (1978), Best Young Player (1958), and Golden Glove (1994) at different points — a 1930 tournament will correctly show fewer award rows than a 2022 one. **Penalty shootouts** only record the outcome (scored or missed) for 2026, not kicking order.

Filtering decisions already made

This project is scoped to **men's tournaments only**, filtered out during the SQL stage. Team names are conformed across both sources through a verified mapping (e.g. Türkiye→Turkey, IR Iran→Iran, Zaire→DR Congo), and West and East Germany are merged into one team, Germany. Four genuinely new 2026 teams (Cabo Verde, Curaçao, Jordan, Uzbekistan) are added as new rows, since they have no historical match to merge into. Historical splits (Czechoslovakia, the Soviet Union, Yugoslavia) are deliberately left unmerged, so every successor country starts with a clean slate. The full reasoning behind each of these calls is covered in `sql_transformation_guide.pdf`.

Why a cloud database (Supabase)

A database on your own laptop can only ever be reached by your own laptop. Supabase gives a real Postgres database running in the cloud, reachable by Python, by Power BI Desktop, and later by anyone who opens your published report. That reachability is what makes the final step of this project — sharing a working report — possible at all.

Understanding the data: what each source actually is

Three sources contributed real data to this project, and only three. Knowing what each one actually is — not just its filename — matters for interpreting anything you later see in a chart.

jfjelstul/worldcup — the historical backbone

Built by an independent researcher, not published by FIFA itself — a carefully compiled secondary source, not the literal official archive. It covers every World Cup, 1930–2022, both men's and women's (this project uses only the men's half), spread across 22 linked files: matches, goals, bookings, penalty kicks, players, squads, managers, referees, awards, and more. Licensed CC-BY-SA 4.0, which means reuse requires giving credit — this project does so throughout. **How to interpret it:** treat it as thorough and well cross-checked, but still a compilation someone assembled after the fact, not a live official feed.

mominullptr/FIFA-World-Cup-2026-Dataset — the 2026 depth layer

Covers only the 2026 tournament, but far more richly than any historical source ever recorded: expected goals (xG), player market values, ELO ratings, possession, shot statistics, referee tendencies. Licensed CC0 (public domain), no attribution required. **How to interpret it:** because these metrics exist for no other tournament, they can only ever describe 2026 in isolation — there's no historical baseline to compare an xG number against, so any claim like "highest xG ever" isn't meaningful with this data. Treat these fields as a 2026 snapshot, not a trend.

The manually added 2026 award winners

The one genuinely hand-entered table in the whole project — 6 rows (Golden Ball, Golden Boot, Silver Boot, Bronze Boot, Golden Glove, Best Young Player), cross-checked against multiple independent news sources after the tournament ended. **How to interpret it:** small, but no less reliable for being manual — it exists precisely because neither GitHub source tracks awards for 2026 at all, not because the other data was insufficient elsewhere.

What was consulted but never became data

Wikipedia's FIFA World Cup records page was read once, partway through this project, purely to decide how to correctly handle historical country splits and reunifications (Germany, Czechoslovakia, the Soviet Union, Yugoslavia, Zaire). It shaped a modeling *decision* — no row of data was ever pulled from it into any table. Every number in the final model still traces back to only the three sources above.

The procedure starts below.

1. What is Supabase, and why are we using it?

Supabase gives you a real Postgres database running in the cloud, for free, in about two minutes. "In the cloud" is the important part: unlike a database installed on your own laptop, a cloud database can be reached from anywhere — your Python script, your Power BI report, and later, anyone you share the finished report with. That's what makes the final step of this project possible: publishing a report other people can actually open.

2. Create your account and organization

1. Go to `supabase.com` and sign up (email, or GitHub/Google sign-in all work).
2. Supabase will ask you to create an **organization** before it lets you create a project — this isn't optional, it's just how Supabase groups projects together. Name it **"Company Standard Data Analysis"**.
3. Choose the **Free** plan when asked.

3. Create your project

Inside your new organization, click **New project** and fill in:

Field	What to enter	Why
Project name	Something clear, e.g. worldcup-analytics-capstone	Just a label — easy to rename later
Database password	Click Generate, then keep clicking until it's letters + numbers only	Avoids a real problem later: special characters like @ # / need extra handling in code
Region	Whichever is closest to you	Matters for speed once Power BI is asking this database live questions

Before clicking Create, also set the security options exactly like this:

- **Automatically expose new tables** — uncheck this. Our tables don't need to be reachable through Supabase's public web API; we connect directly with a username and password instead.
- **Enable automatic RLS** — leave unchecked. Row Level Security is for apps with untrusted public users; we're the only ones querying this database directly, so it isn't needed.

Click **Create new project**, then wait a minute or two while it provisions.

IMPORTANT: copy the database password somewhere safe the moment it's shown. Supabase will not show it to you again in full.

4. Get your 5 connection values

Every later step — Python, SQL, Power BI — needs the same five pieces of information to reach your database. Here's where to find them:

1. Click the green **Connect** button (top of your project page).
2. Click the **Direct** tab (the one with a database icon, not Framework/Server/ORM/MCP).

3. Choose the **Session pooler** option, not "Direct connection" and not "Transaction pooler".
4. Scroll down to where it lists `host`, `port`, `database`, and `user` as separate fields — these, plus your saved password, are your 5 values.

> **Why Session pooler specifically?** The "Direct connection" option defaults to a network format (IPv6) that many tools outside your own laptop can't reliably reach — including, later, Power BI's published report. Session pooler avoids that problem and is what the rest of this project is built around.

Write your 5 values down now, in this format:

```
HOST:     (from the Session pooler screen)
PORT:     5432
DATABASE: postgres
USER:     postgres.<your-own-project-ref>
PASSWORD: (the one you saved in step 3)
```

5. Set up your project folder

1. Create a folder on your computer for this project — anywhere you like, e.g. `Documents/FIFA World Cup Project`.
2. Put `ingest_raw.py` and `.env.example` inside it (both provided separately).
3. Rename `.env.example` to `.env`, open it, and replace the placeholder values with your own 5 values from Step 4.

> **Never share your .env file.** Not in a group chat, not on GitHub, not pasted into any AI tool. It contains your database password in plain text. The whole reason this project keeps credentials in a separate file instead of typing them into the scripts is so the scripts themselves stay safe to share — but that only works if .env itself stays private.

6. Open Command Prompt at your project folder

Everything from here happens in **Command Prompt** (cmd) — the text-based way to tell your computer to run a program, instead of clicking icons. We use it because `pip` (which installs libraries) and `python` (which runs scripts) are both commands, not apps with buttons.

This works the same way for any project folder, not just this one:

1. Open **File Explorer** and navigate into your project folder (the one containing `ingest_raw.py` and `.env`).
2. Click once on the **address bar** at the top of the window (where the folder path is shown) — this selects the whole path.
3. Type `cmd` and press **Enter**.
4. A black Command Prompt window opens, already pointed at your project folder — you'll see the folder path right before the blinking cursor. That's what tells you it's in the right place.

> **Why this matters:** every command you type next runs "from" wherever Command Prompt is currently pointed. Opening it directly inside your project folder means Python can find `.env` and `ingest_raw.py` sitting right next to it, without you having to type out the full folder path every time.

7. Install the Python libraries (once)

In that same Command Prompt window, type this and press Enter:

```
pip install pandas sqlalchemy psycopg2-binary python-dotenv
```

What this actually does: Python doesn't come with everything built in. This command reaches out to the internet and downloads four small toolkits your script needs — one to handle spreadsheet-shaped data (pandas), two to talk to the Postgres database (sqlalchemy, psycopg2), and one to read your .env file (python-dotenv). You'll see a handful of "Successfully installed" lines when it's done.

You only need to do this once per computer, not once per project.

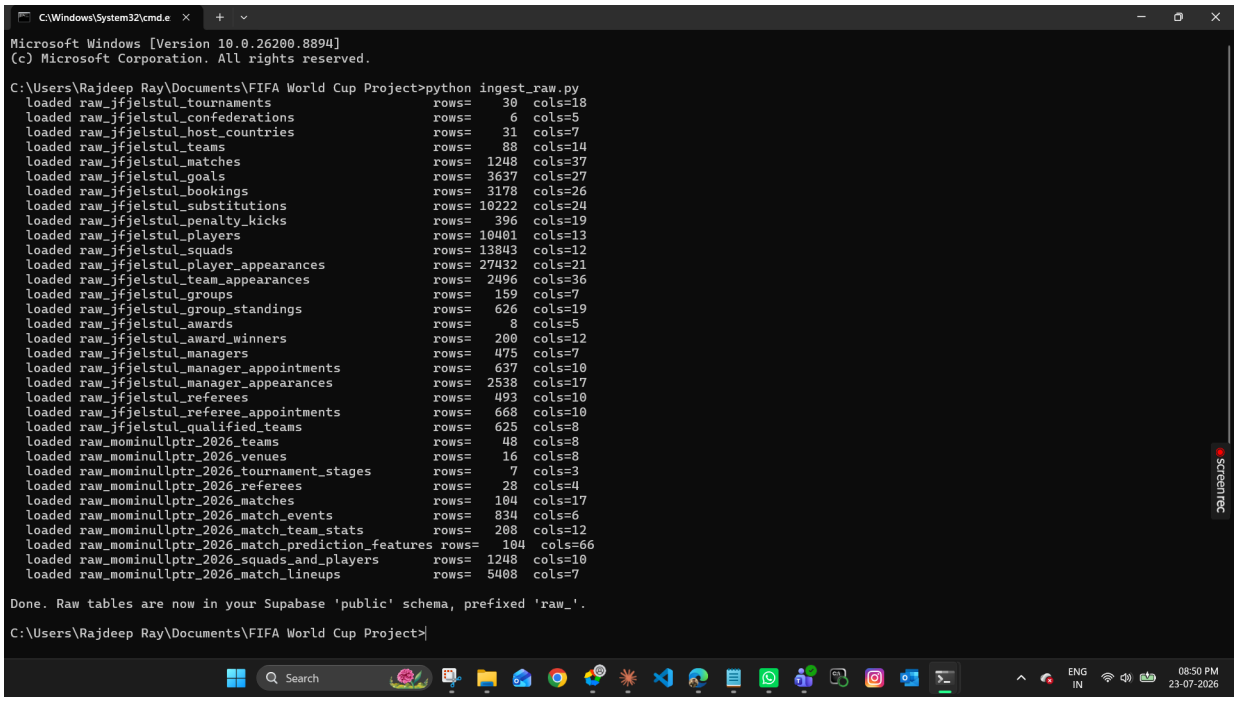
8. Run the script

Still in the same window, type:

```
python ingest_raw.py
```

What this actually does: this is the command that starts everything. It tells Python "run the instructions written inside ingest_raw.py, right now." That script then reads your .env file to find your database, reaches out to two GitHub repositories, downloads 32 spreadsheets of real FIFA World Cup data, and writes each one into your Supabase database as its own table.

You'll see one line print per file as it loads. Here's a real, confirmed run of this exact script:



A real successful run of ingest_raw.py — all 32 tables loaded, no errors.

Once you see the "Done" line, this step is complete: you have a real cloud database, filled with real historical World Cup data.

> **If something goes wrong here**, it's almost always one of two things: your .env file has a typo in one of the 5 values, or your database password wasn't saved correctly. Double-check both against what Supabase showed you in Step 4 before trying again. It is completely safe to run `python ingest_raw.py` again as many times as you need — it simply overwrites the tables each time.

9. Load the verified 2026 awards — this is where the guide ends

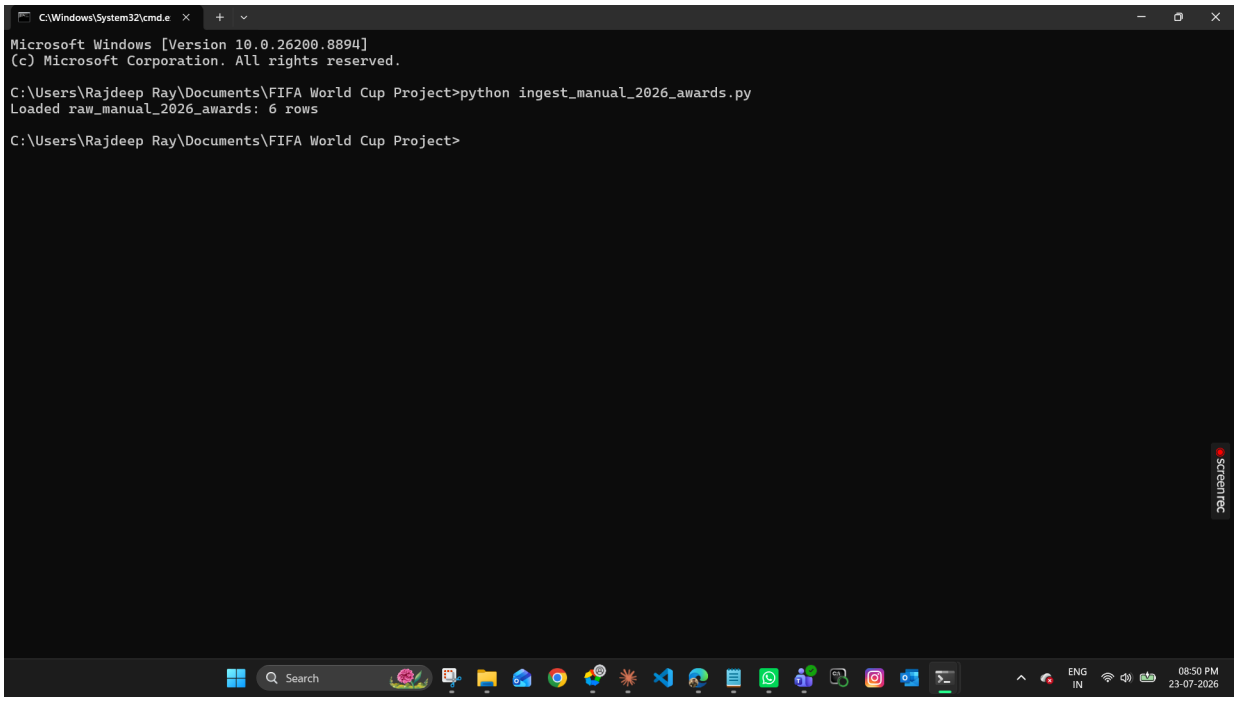
There's one small piece of real World Cup data that doesn't exist in either GitHub source: who actually won the 2026 awards (Golden Ball, Golden Boot, Golden Glove, and so on). Rather than leave that gap, a second small script adds these 6 award winners by hand — hand-verified against multiple news sources after the tournament ended, not guessed.

This script follows the exact same pattern as the first one (same .env, same connection method), it just writes 6 rows we typed ourselves instead of downloading a CSV.

In the same Command Prompt window, run:

```
python ingest_manual_2026_awards.py
```

A real, confirmed run of this script looks like this:

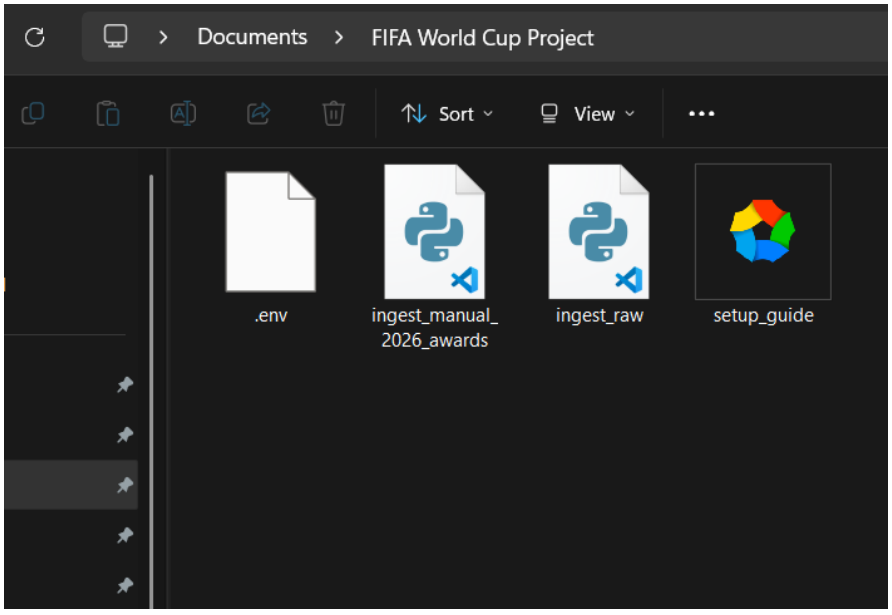


A real successful run of ingest_manual_2026_awards.py — 6 rows loaded.

Once you see `Loaded raw_manual_2026_awards: 6 rows`, your database now holds everything this project needs, in raw form: 32 tables from GitHub, plus this one small hand-verified addition — 33 raw tables in total.

Checkpoint: does your folder look like this?

Before moving on, your project folder should contain exactly these 4 items — nothing more needed yet:



A real project folder at this checkpoint: .env, ingest_manual_2026_awards.py, ingest_raw.py, and this setup guide, sitting side by side.

- `.env` — your filled-in credentials (Step 5)
- `ingest_raw.py` — the script from Step 8

- `ingest_manual_2026_awards.py` — the script from Step 9
- `setup_guide.pdf` — this guide itself, kept for reference

If your folder matches this, you're in the right place. That's the end of this setup guide. The next stage of the project turns these raw tables into a clean, modeled set of tables using SQL — covered in `sql_transformation_guide.pdf`, the next document in this series.
